

UNIVERSITATEA TEHNICĂ A MOLDOVEI
FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ

Disciplina: Tehnici de programare
Lucrarea practică nr. 4.

Elaborat: Turcu Anastasia;
Grupa: SI-264;

Verificat: Reițman Patricia.

Chișinău, 2026

Capitolul 1 CODEDEX. Hello Word

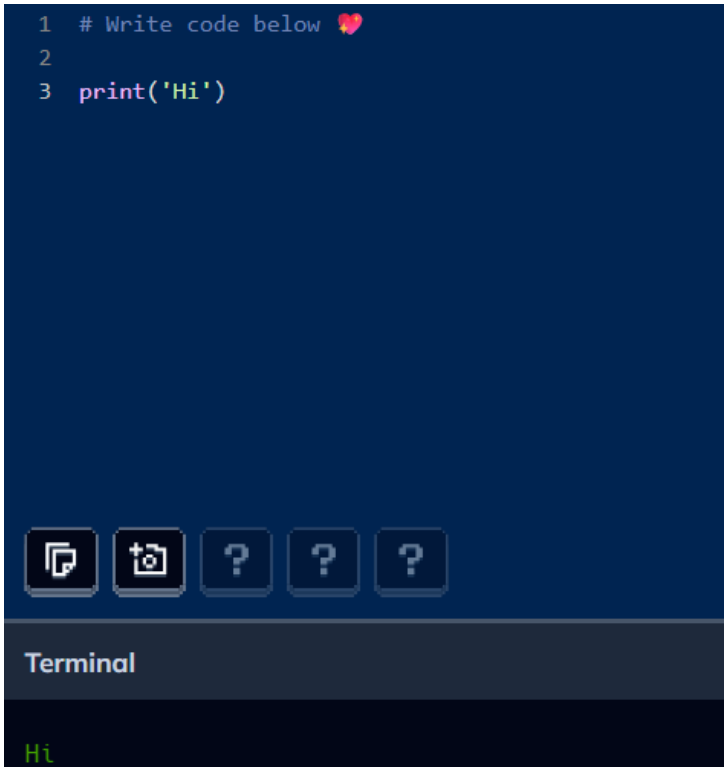


fig. 1.1 - Setting up

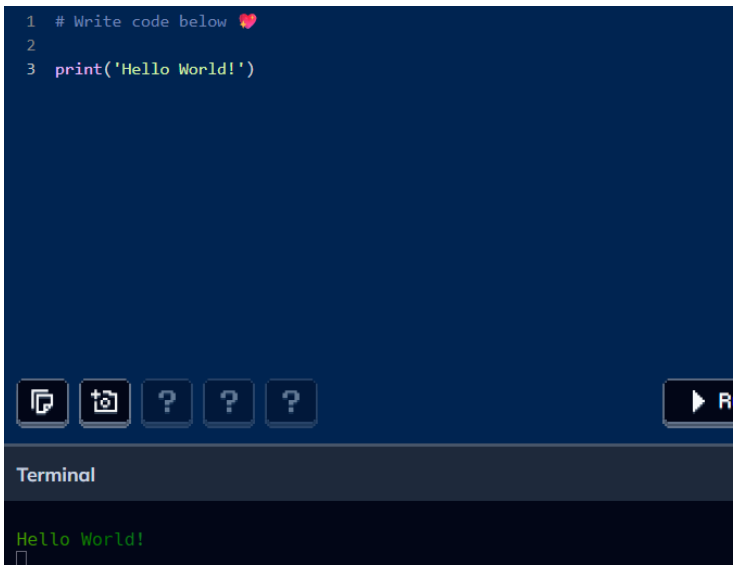


fig. 1.2 - Hello Word



fig. 1.3 - Pattern



fig. 1.4 - Initials

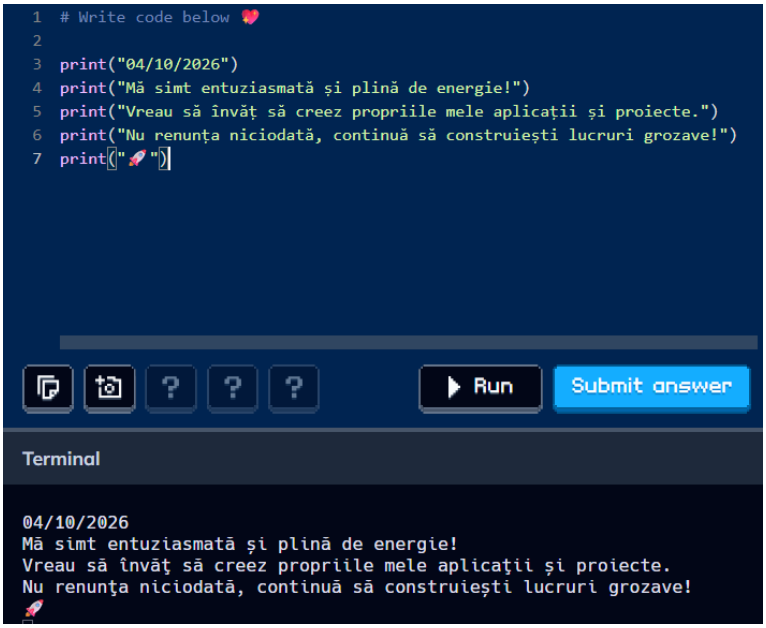


fig. 1.5 - Snail Mail

Capitolul 2 CODEDEX. Variables

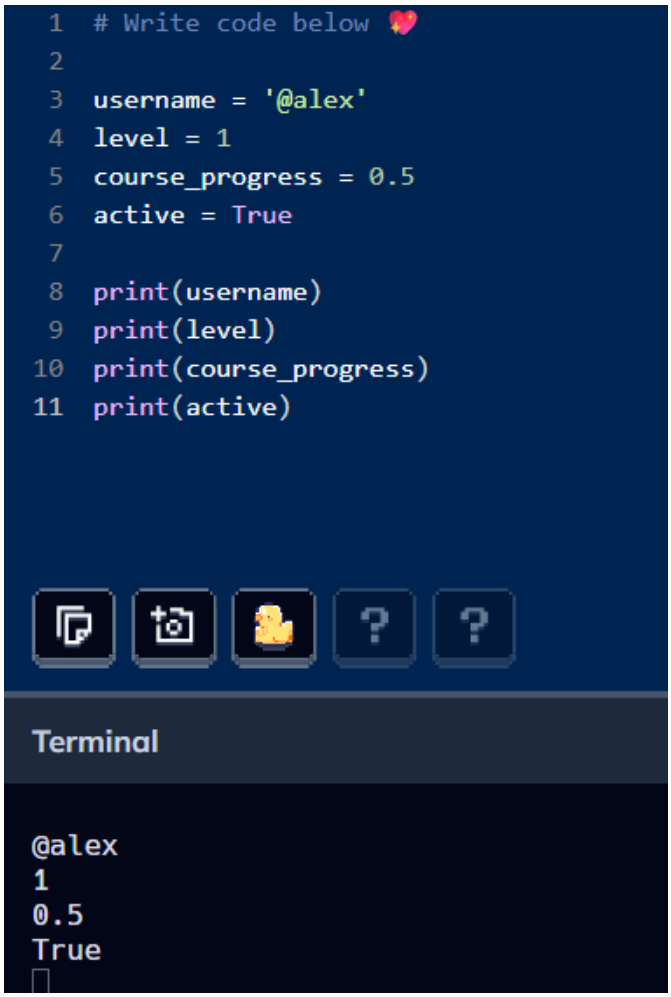


fig. 2.1 - Data Types

```
# Write code below 💔

fahrenheit = 55
celsius = (fahrenheit - 32) / 1.8

print(round(celsius,2))
```

Terminal

```
12.78
```

fig. 2.2 - Temperature

```
# Write code below 💔

mass = 70
height = 1.75

bmi = mass / (height ** 2)

print(round(bmi,2))
```

Run

Terminal

```
22.86
```

fig. 2.3 - BMI



fig. 2.4 - Pythagorean



fig. 2.5 Currency

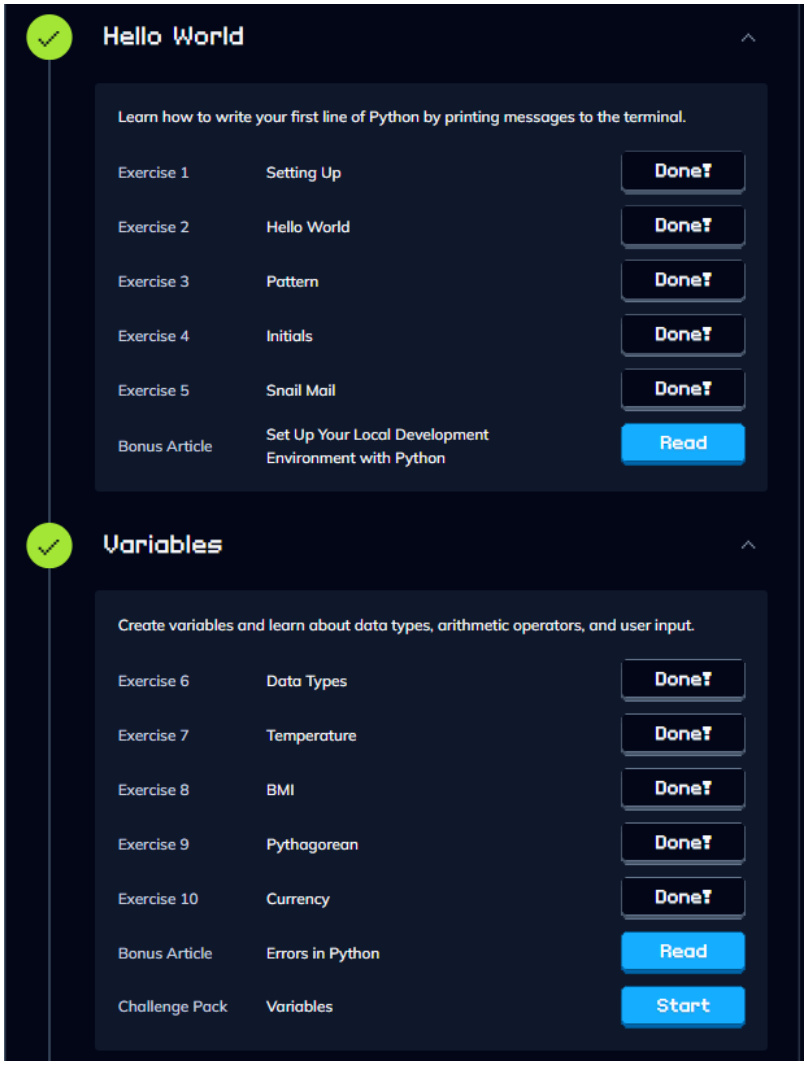


fig. 2.6 - Rezolvarea tuturor exercițiilor

Capitolul 3 LEARN PYTHON. Learn the basics.

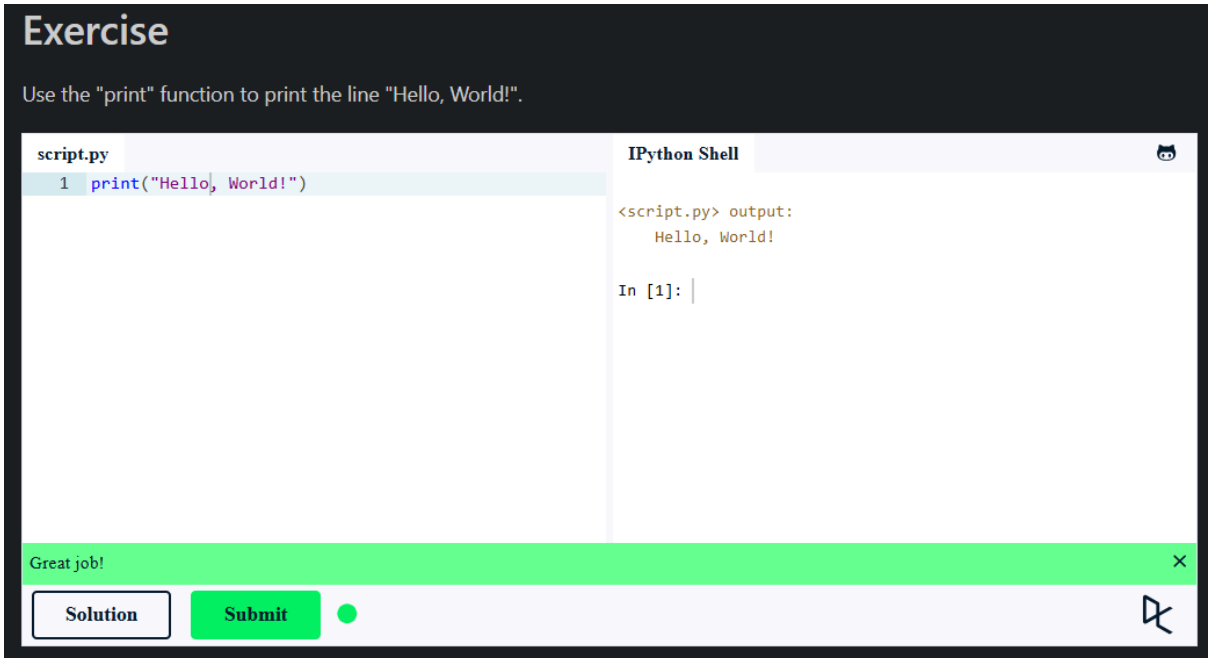


fig. 3.1 - Hello, Word

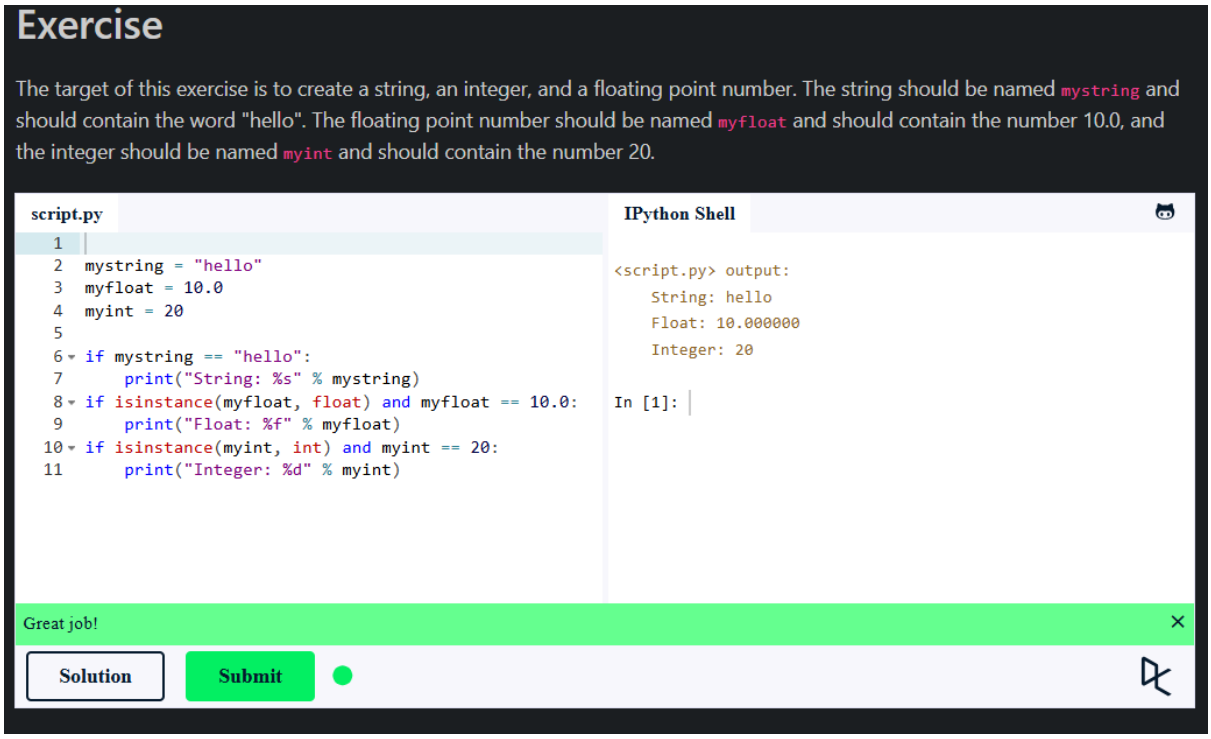


fig. 3.2 - Variables and Types

Exercise

In this exercise, you will need to add numbers and strings to the correct lists using the "append" list method. You must add the numbers 1,2, and 3 to the "numbers" list, and the words 'hello' and 'world' to the strings variable. Python list tutorial

You will also have to fill in the variable second_name with the second name in the names list, using the brackets operator `[]`. Note that the index is zero-based, so if you want to access the second item in the list, its index will be 1.

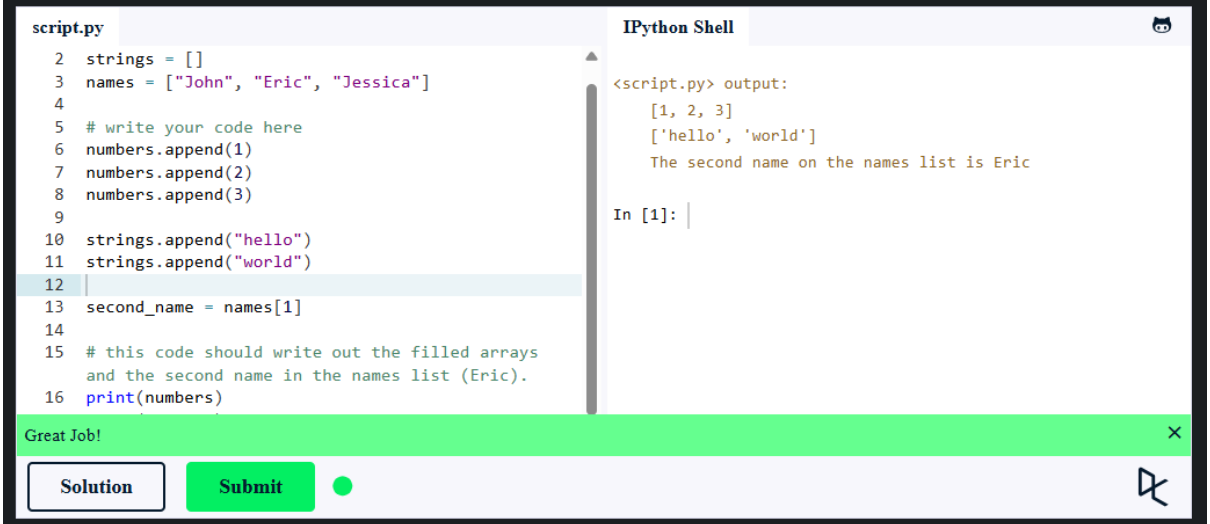


fig. 3.3 - Lists

Exercise

The target of this exercise is to create two lists called `x_list` and `y_list`, which contain 10 instances of the variables `x` and `y`, respectively. You are also required to create a list called `big_list`, which contains the variables `x` and `y`, 10 times each, by concatenating the two lists you have created.

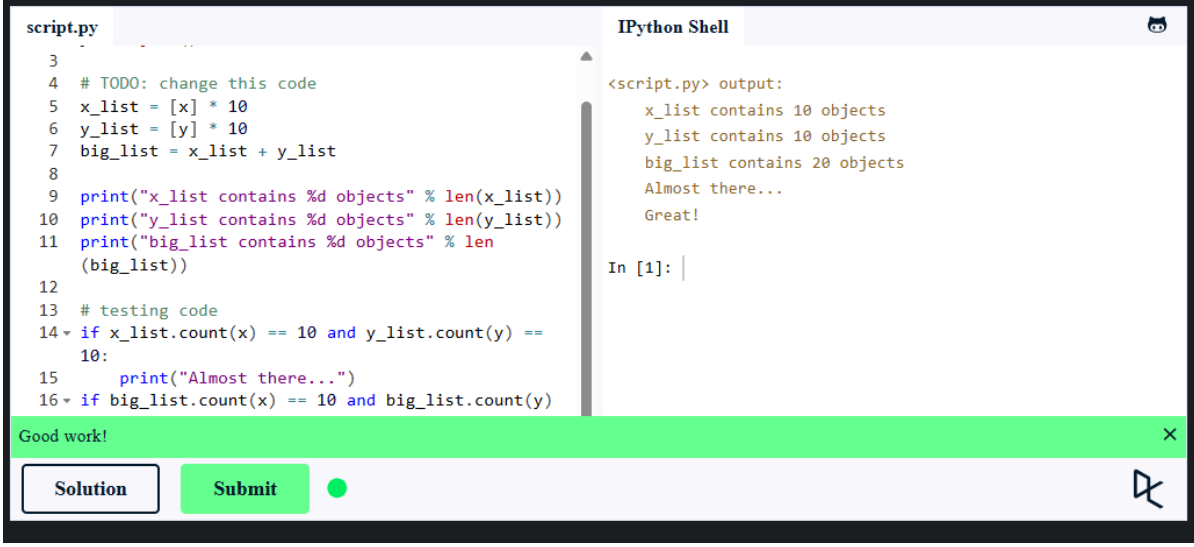


fig. 3.4 - Basic Operators

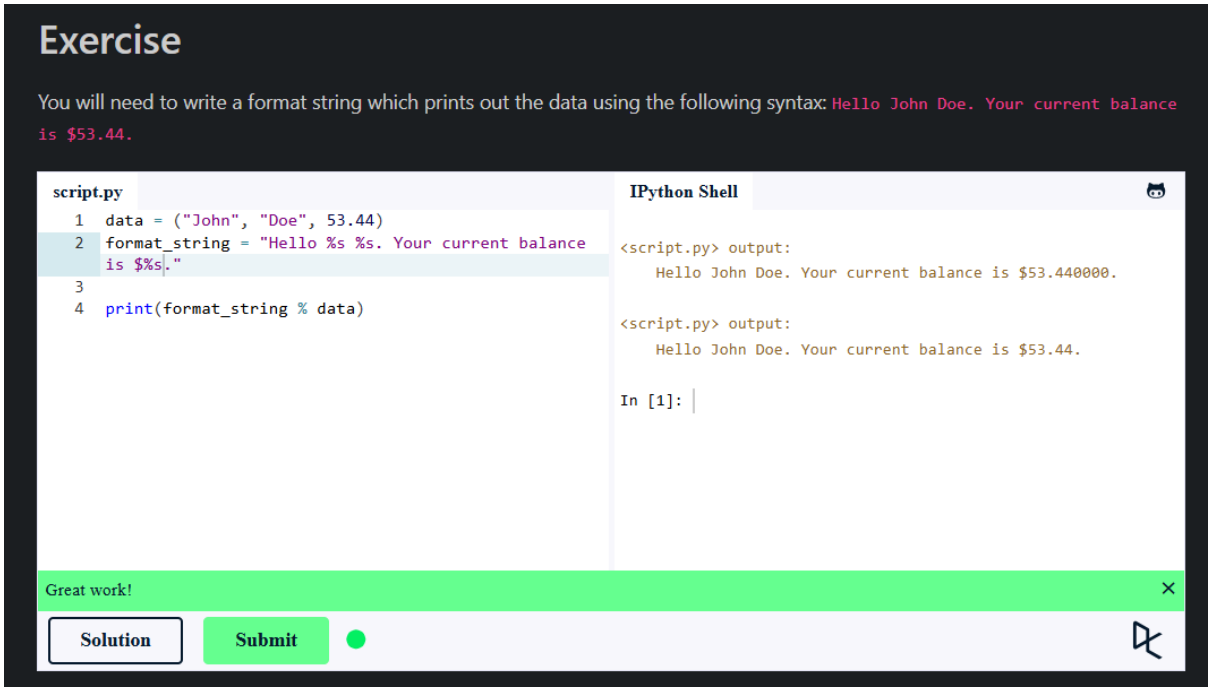


fig. 3.5 - String Formatting

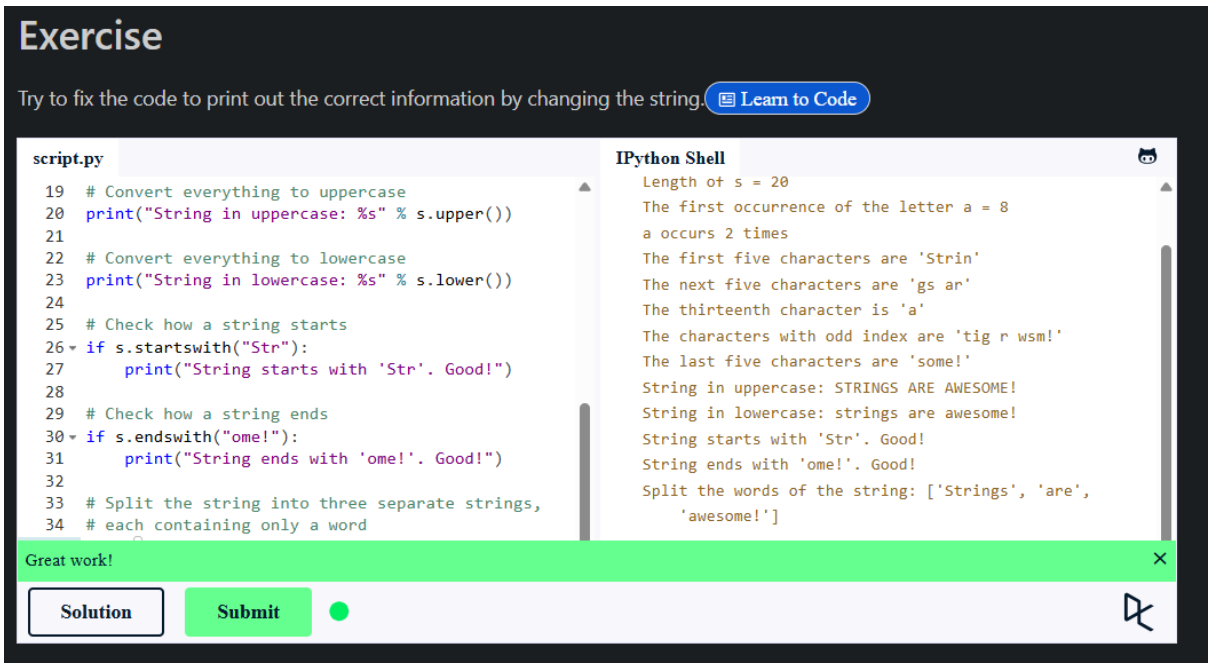


fig. 3.6 - Basic String Operations

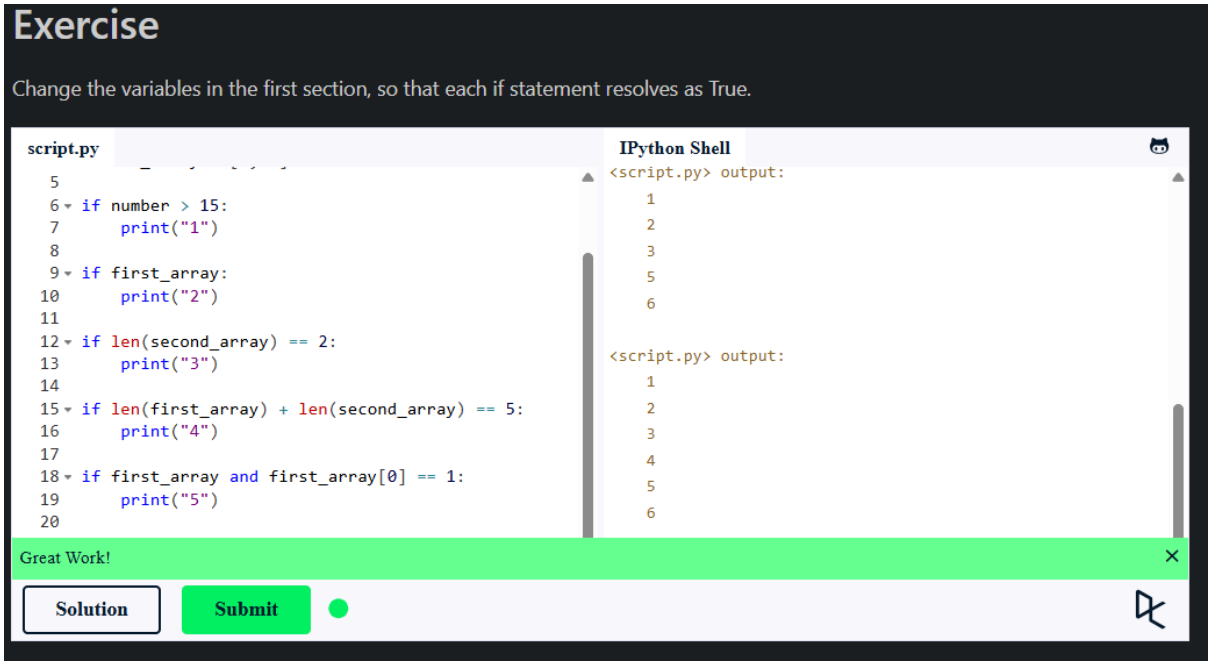


fig. 3.7 - Conditions

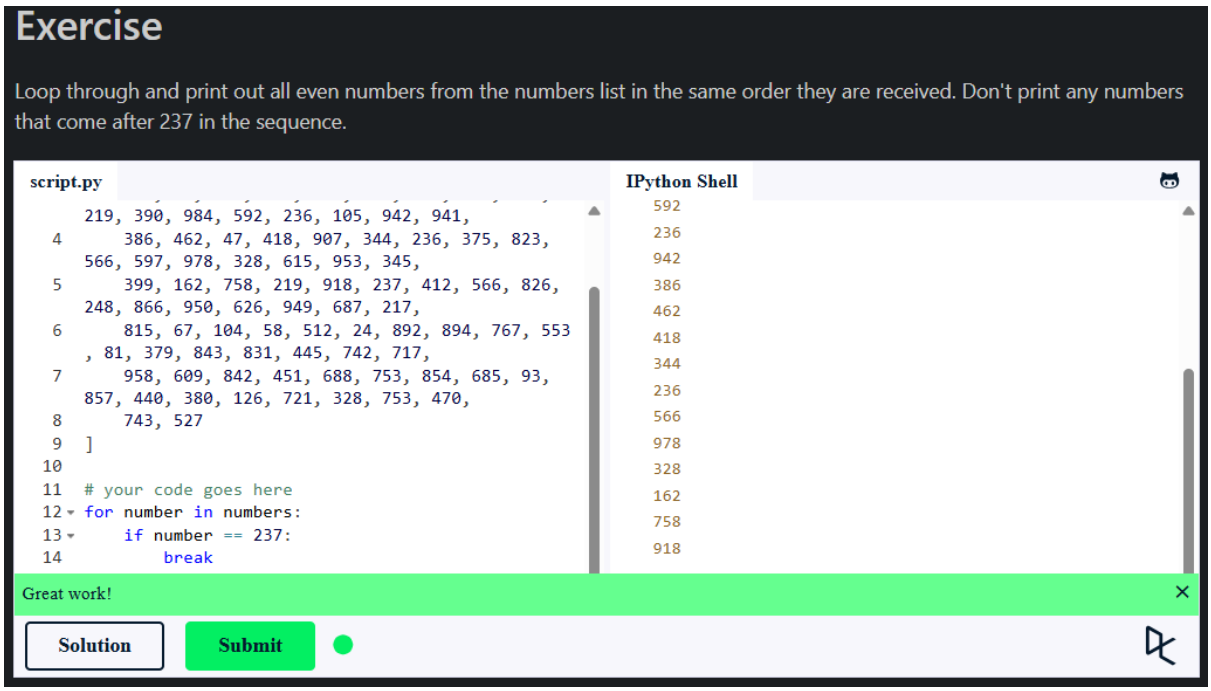


fig. 3.8 - Loops

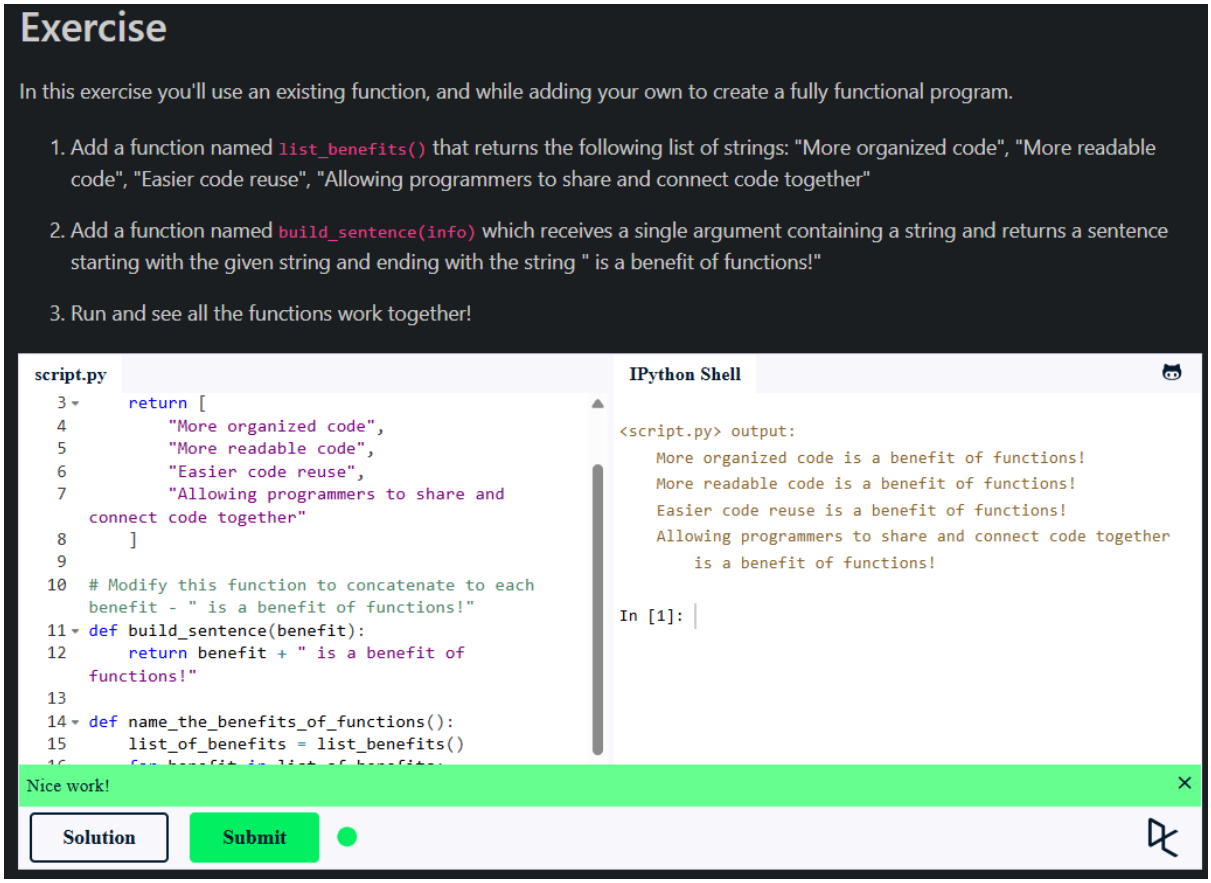


fig. 3.9 - Functions



fig. 3.10 - Classes and Objects

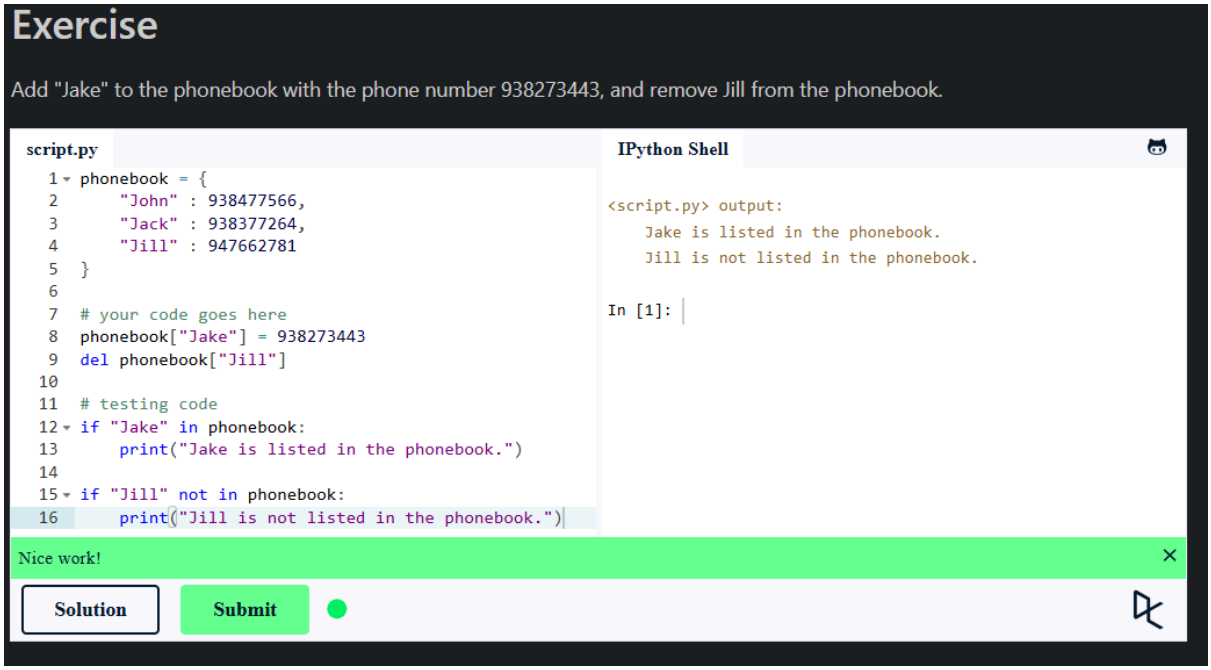


fig. 3.11 - Dictionaries

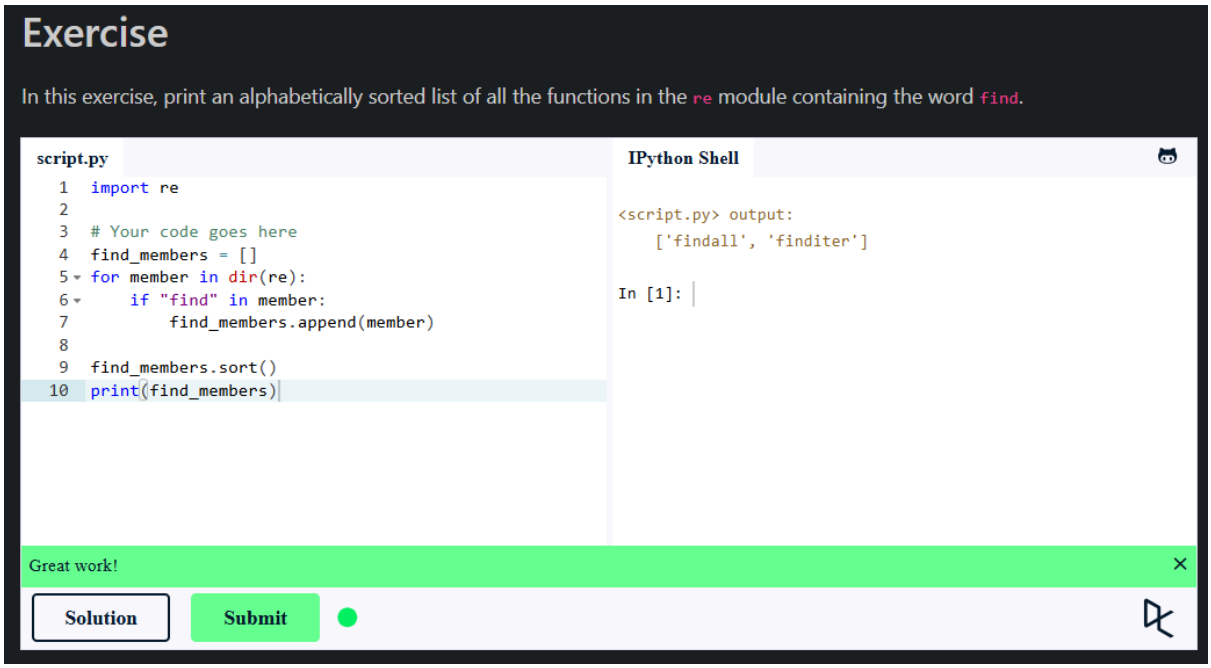


fig. 3.12 - Modules and Packages

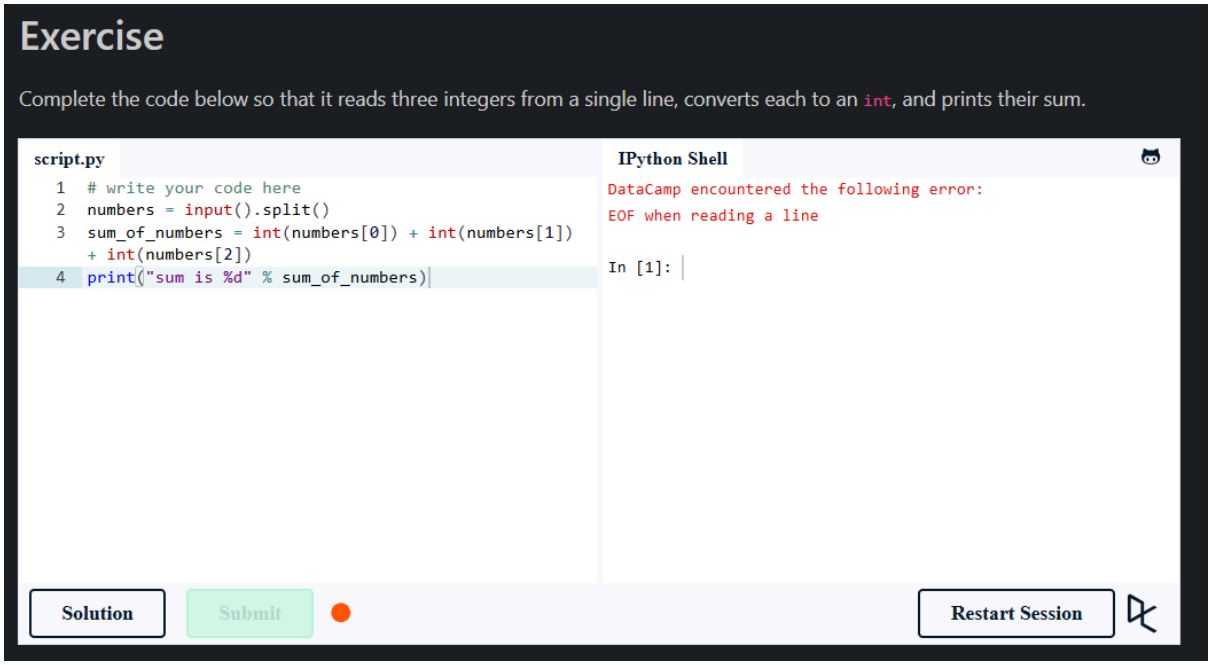


fig. 3.13 - Input and Output

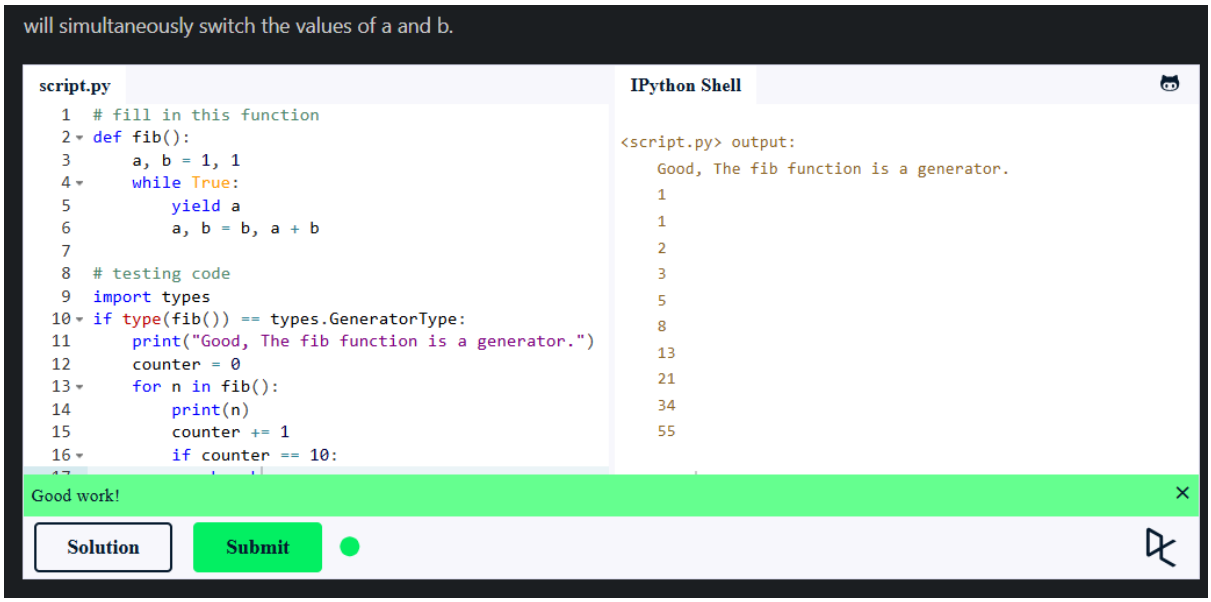


fig. 4.1 - Generators

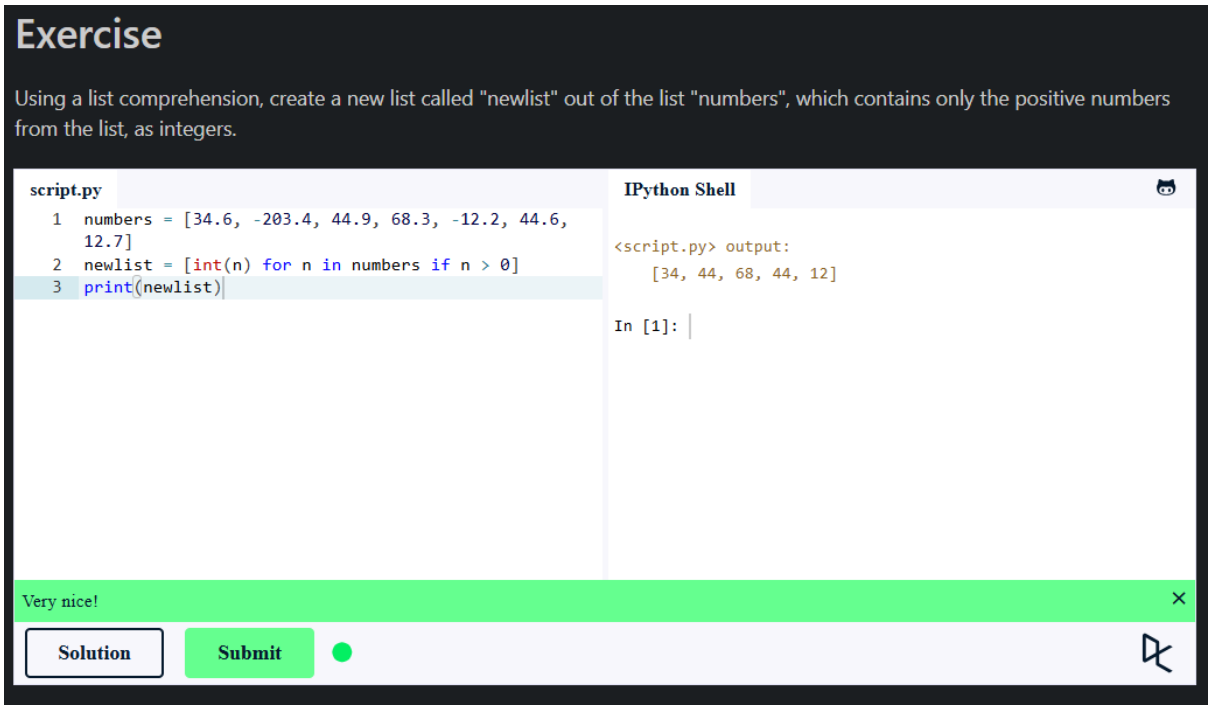


fig. 4.2 - List Comprehensions

Exercise

Write a program using lambda functions to check if a number in the given list is odd. Print "True" if the number is odd or "False" if not for each element.

script.py

```
1  l = [2, 4, 7, 3, 14, 19]
2  for i in l:
3      print((lambda x: x % 2 != 0)(i))
```

IPython Shell

```
<script.py> output:
    False
    False
    True
    True
    False
    True

In [1]:
```

Nice work!

Solution | Submit

fig. 4.3 - Lambda Functions

Exercise

Fill in the `foo` and `bar` functions so they can receive a variable amount of arguments (3 or more) The `foo` function must return the amount of extra arguments received. The `bar` must return `True` if the argument with the keyword `magicnumber` is worth 7, and `False` otherwise.

script.py

```
1  # edit the functions prototype and implementation
2  def foo(a, b, c, *args):
3      return len(args)
4
5  def bar(a, b, c, **kwargs):
6      return kwargs.get("magicnumber") == 7
7
8  # test code
9  if foo(1, 2, 3, 4) == 1:
10     print("Good.")
11 if foo(1, 2, 3, 4, 5) == 2:
12     print("Better.")
13 if bar(1, 2, 3, magicnumber=6) == False:
14     print("Great.")
15 if bar(1, 2, 3, magicnumber=7) == True:
16     print("Awesome!")
```

IPython Shell

```
<script.py> output:
    Good.
    Better.
    Great.
    Awesome!

In [1]:
```

Great work!

Solution | Submit

fig. 4.4 - Multiple Function Arguments

An example regex is `r"^(From|To|Cc).*?python-list@python.org"` Now for an explanation: the caret `^` matches text at the beginning of a line. The following group, the part with `(From|To|Cc)` means that the line has to start with one of the words that are separated by the pipe `|`. That is called the OR operator, and the regex will match if the line starts with any of the words in the group. The `.*?` means to un-greedily match any number of characters, except the newline `\n` character. The un-greedy part means to match as few repetitions as possible. The `.` character means any non-newline character, the `*` means to repeat 0 or more times, and the `?` character makes it un-greedy.

So, the following lines would be matched by that regex: `From: python-list@python.org To: !asp]<,. python-list@python.org`

A complete reference for the re syntax is available at the python docs.

As an example of a "proper" email-matching regex (like the one in the exercise), see this



fig. 4.5 -Regular Expressions

Exercise

Handle all the exception! Think back to the previous lessons to return the last name of the actor.



fig. 4.6 - Exception Handling

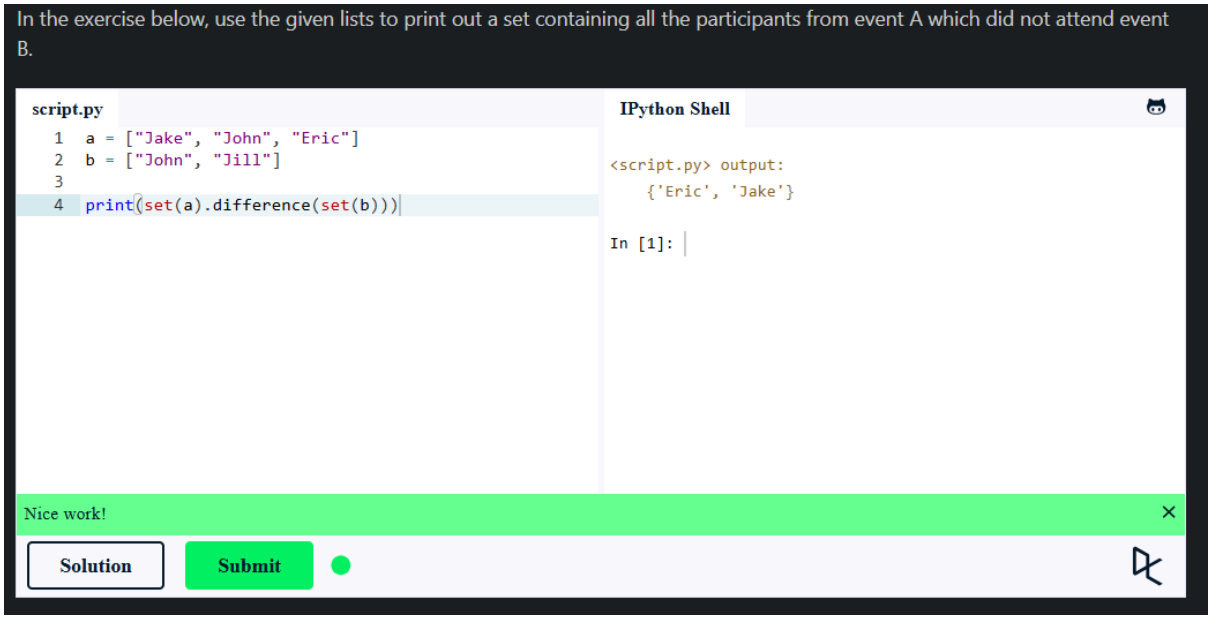


fig. 4.7 - Sets

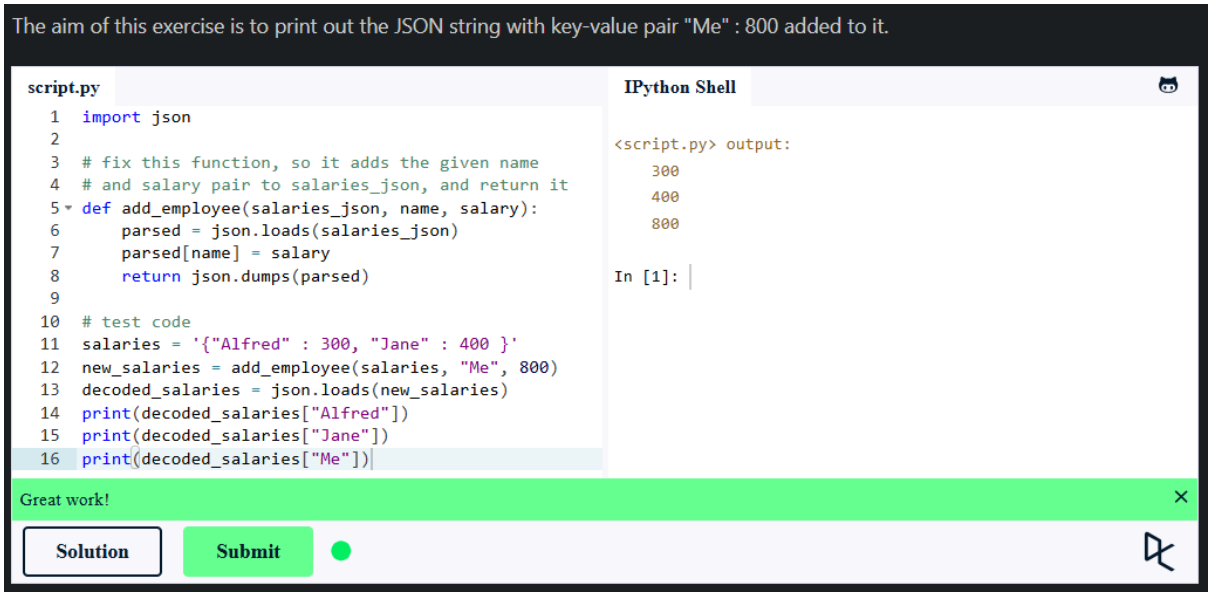


fig. 4.8 - Serialization

Concluzie

În concluzie, parcursul de la primele linii de cod în Python până la dezvoltarea funcționalităților interactive a marcat o trecere importantă de la teorie la practica algoritmică. Fiecare etapă a proiectului de la gestionarea excepțiilor la rezolvarea erorilor întâmpinate m-a învățat să fiu mai atent la detalii, să îmi testez logica pas cu pas și să caut soluții eficiente. Am înțeles că programarea înseamnă răbdare, creativitate și capacitatea de a aborda problemele complexe prin pași simpli. Această experiență îmi oferă o bază solidă și motivația necesară pentru a continua să explorez domeniul tehnologiei.